SMRTTECH 4AI3 - Artificial Intelligence and Machine Learning

Lab 7 - Support Vector Machines (SVM) and Classification

Instructor: Siqi Zhao

Due: 11:59 PM, the day before your next lab session

Lab Instructions

Please read and follow these instructions:

- Submit your lab on Avenue to Learn (A2L) before the deadline (11:59 PM on the day before your next lab session). The submission will close after the deadline.
- Complete all lab exercises independently. Do not copy code or solutions from other students. If two lab submissions (or portions of them) are found to be identical due to copying, both submissions will receive a grade of zero.
- You may use Google Colab or any other Python programming environment of your choice.
- Read each question carefully. Use AI or machine learning libraries only when specifically instructed. Include comments in your code where necessary.
- Submit your working Python source files (.py) or Jupyter notebooks (.ipynb). Do not submit ZIP files.
- If you use Google Colab, include your notebook link in the submission comments on A2L. Ensure the sharing settings are set to "Anyone with the link."

Lab Exercises

1. **Import the required Python packages.**

   Import at least the following packages:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

   You may import additional packages as needed to complete the exercises.

2. **Generate a synthetic dataset using make_moons.**

   Use the `make_moons()` function to generate a two-dimensional classification dataset.

   Assign the generated feature values to `X` and the corresponding class labels to `y`.

```
X, y = make_moons(
    n_samples=400,
    noise=0.15,
    random_state=42
)
```

   The parameter `n_samples` determines the total number of generated data points.

   The parameter `noise` controls the standard deviation of Gaussian noise added to the generated data.

3. **Explore the dataset.**

   Display the first five instances of `X` and `y` to understand the structure of the dataset.

```
print(X[:5])
print(y[:5])
```

   Briefly describe the number of features and classes in the dataset.

4. **Visualize the dataset.**

   Generate a scatter plot of the two-dimensional dataset.

   Use different colors to distinguish the data points belonging to different classes.

   Your plot should include:

   - A descriptive title.
   - Labels for both axes.
   - A legend indicating the classes.

5. **Split the dataset into training, validation, and testing sets.**

   Use the `train_test_split()` function to divide the dataset into the following subsets:

   - Training set: 60% of the dataset.
   - Validation set: 20% of the dataset.
   - Testing set: 20% of the dataset.

   Use a fixed `random_state` value to ensure reproducibility.

   You may perform the split in two stages using `train_test_split()`.

   Print the number of instances in each subset.

6. **Standardize the dataset using StandardScaler.**

   Support Vector Machine models are sensitive to the scale of input features. Therefore, standardize the dataset before training the SVM models.

   Use `StandardScaler` to standardize the training, validation, and testing datasets.

   - Fit the scaler using only the training data.

- Transform the training data using the fitted scaler.
- Apply the same scaler to the validation and testing datasets.

Note: Do not fit the scaler separately on the validation or testing datasets.

7. **Train and compare three SVM classifiers.**

   Create three Support Vector Machine classification models using the `SVC()` class with the following kernels:

   - Linear kernel: `linearSVM_clf`
   - Polynomial kernel: `polynomialSVM_clf`
   - Radial Basis Function (RBF) kernel: `rbfSVM_clf`

   Train all three classifiers using the standardized training dataset and the corresponding training labels.

   You may experiment with different hyperparameter values, such as:

   - `C`
   - `degree` (for the polynomial kernel)
   - `gamma` (for polynomial and RBF kernels)

   Use the validation dataset to evaluate and compare the performance of the three models.

   Print the classification accuracy of each model using the `score()` method.

8. **Evaluate the best SVM classifier.**

   Based on the validation results from Question 7, select the SVM model with the highest validation accuracy.

   Evaluate the selected model using the standardized testing dataset.

   Print the final testing accuracy using the `score()` method.

   Briefly answer the following questions:

   (a) Which SVM kernel achieved the highest validation accuracy?

   (b) How does the testing accuracy compare with the validation accuracy?

   (c) Why might a nonlinear SVM kernel perform better than a linear kernel on the `make_moons` dataset?